

CSE4203: Computer Graphics
Lecture – 8 (Part - A)
Fractal Geometry

Outline

- Fractal
- Fractal Dimension
- The Cantor Set
- Koch Snowflake
- Sierpinski Triangle
- Mandelbrot Set

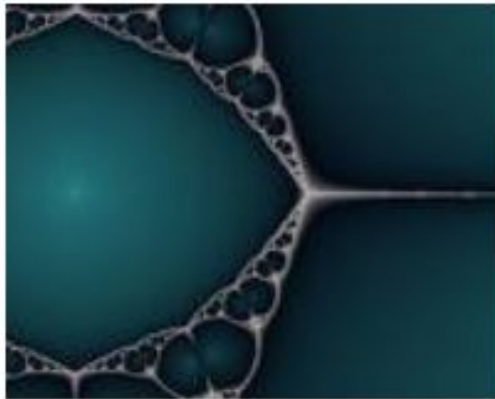
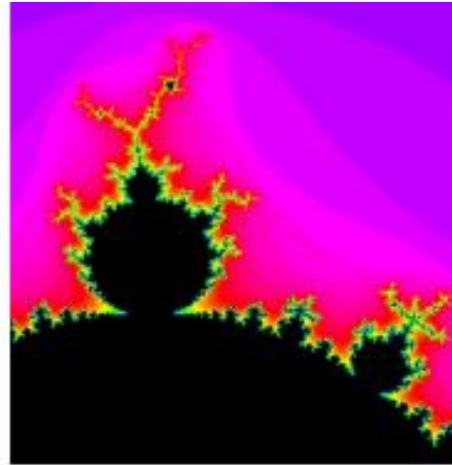
Fractal

- A **Fractal** is a geometric shape generated using set of recursive rules
- Fractal share a **self-similarity** property where part of fractal resembles the whole fractal
- The property is often seen in nature, e.g clouds, plants and landscapes etc.
- We will look at methods used to generate some popular fractals in this lecture

Example of Fractals



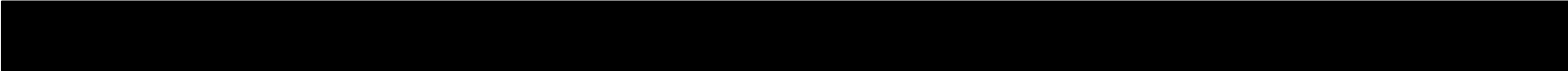
Artificial Fractals



The Cantor Set (1/6)

- A simplest example of a fractal is a Cantor Set devised by George Cantor
- Starting with a straight line at $n = 0$, we remove middle third of each line at each iteration
- As n gets larger, line segments gets smaller

The Cantor Set (2/6)

- A simplest example of a fractal is a Cantor Set devised by
- $n = 0$ 

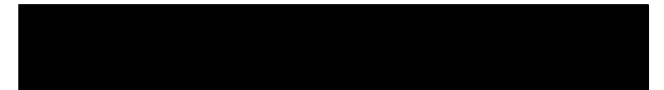
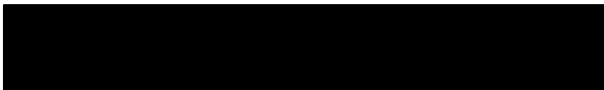
The Cantor Set (3/6)

- A simplest example of a fractal is a Cantor Set devised by

- $n = 0$



- $n = 1$



The Cantor Set (4/6)

- A simplest example of a fractal is a Cantor Set devised by

- $n = 0$ 
- $n = 1$ 
- $n = 2$ 

The Cantor Set (5/6)

- A simplest example of a fractal is a Cantor Set devised by



The Cantor Set (6/6)

- A simplest example of a fractal is a Cantor Set devised by



Fractal Dimension

- The fractal dimension is a measure of the complexity of a fractal
- Let N_n be the number of new elements added at iteration n and ϵ be the scaling factor of the size of new elements then the fractal dimension is

$$D = -\lim_{n \rightarrow \infty} \frac{\log(N_n)}{\log(\epsilon_n)}$$

- For a 2D fractal $D \in [1, 2]$ and for a 3D fractal $D \in [2, 3]$

Dimension of Cantor Set (1/2)



Line segments is
related to iterations

iterations	no. line segments	length of segment
0	1	1
1	2	$\frac{1}{3}$
2	4	$\frac{1}{9}$
3	8	$\frac{1}{27}$
$\vdots$	$\vdots$	$\vdots$
n	2^n	3^{-n}

Dimension of Cantor Set (2/2)

Using

$$D = -\lim_{n \rightarrow \infty} \frac{\log(N_n)}{\log(\epsilon_n)}$$

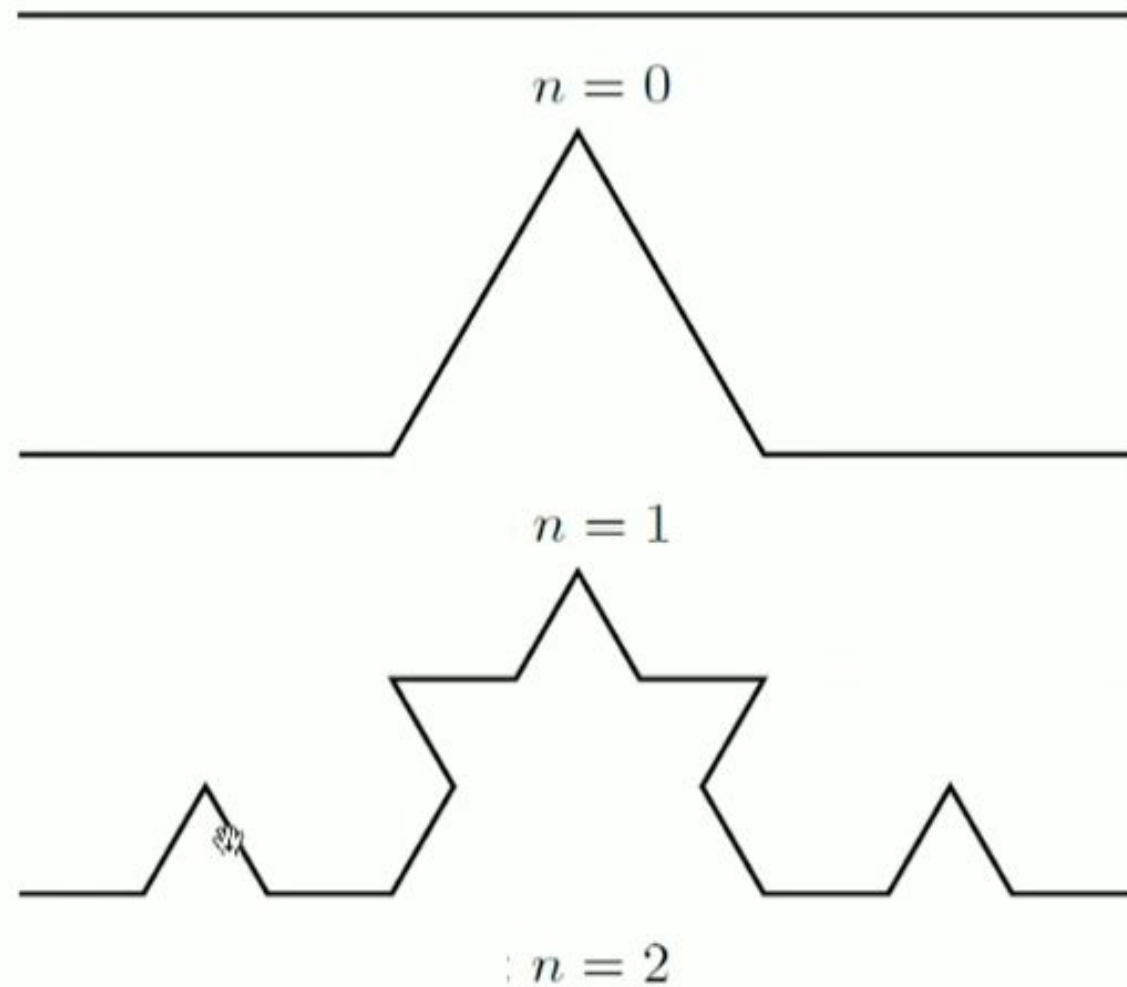
then the fractal dimension of Cantor set is

$$\begin{aligned} D &= -\lim_{n \rightarrow \infty} \frac{\log(2^n)}{\log \frac{1}{3^n}} = -\lim_{n \rightarrow \infty} \frac{n \log 2}{n \log \frac{1}{3}} = -\lim_{n \rightarrow \infty} \frac{\log 2}{\log 3^{-1}} \\ &= \lim_{n \rightarrow \infty} \frac{\log 2}{\log 3} \\ &= 0.6309 \dots \end{aligned}$$

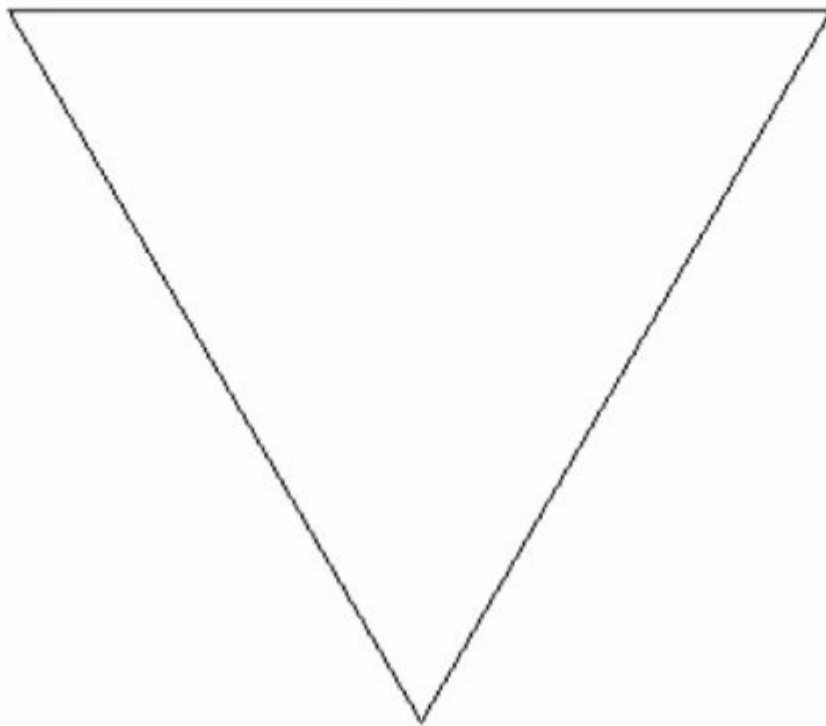
The Koch Snowflake (1/7)

- The Koch Snowflake named after Helge von Koch is a shape with a finite area bounded by infinite circumference
- The rules for generating a Koch Snowflake start with an equilateral triangle at stage = 0
 - Divide each line into three lines of equal length
 - Place an equilateral triangle so that the base fits along the middle length
 - Remove the middle length that is base of the triangle

The Koch Snowflake (2/7)

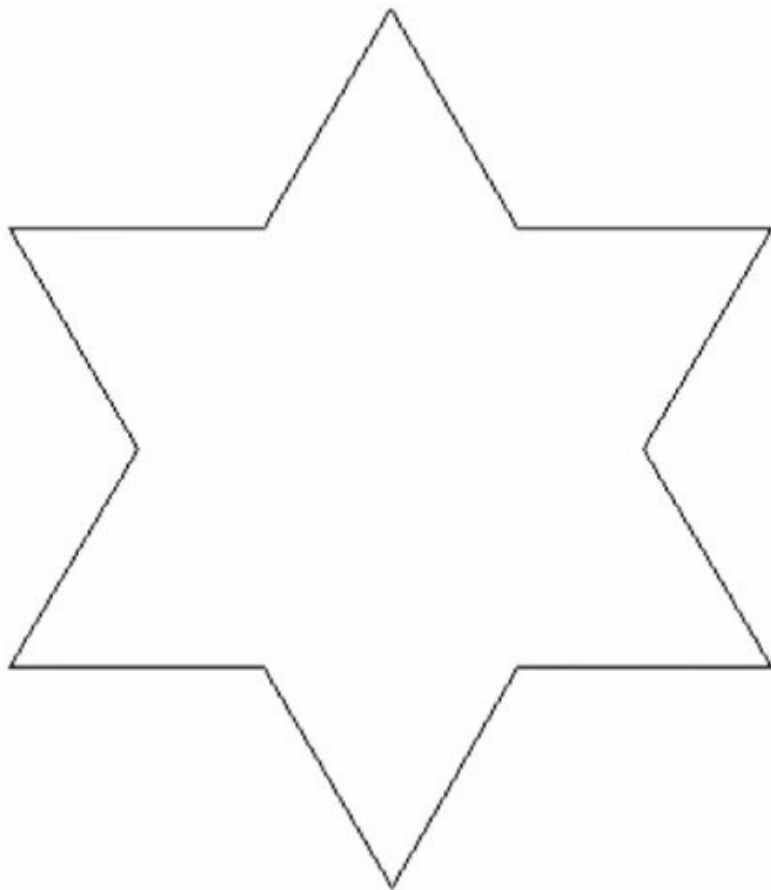


The Koch Snowflake (3/7)



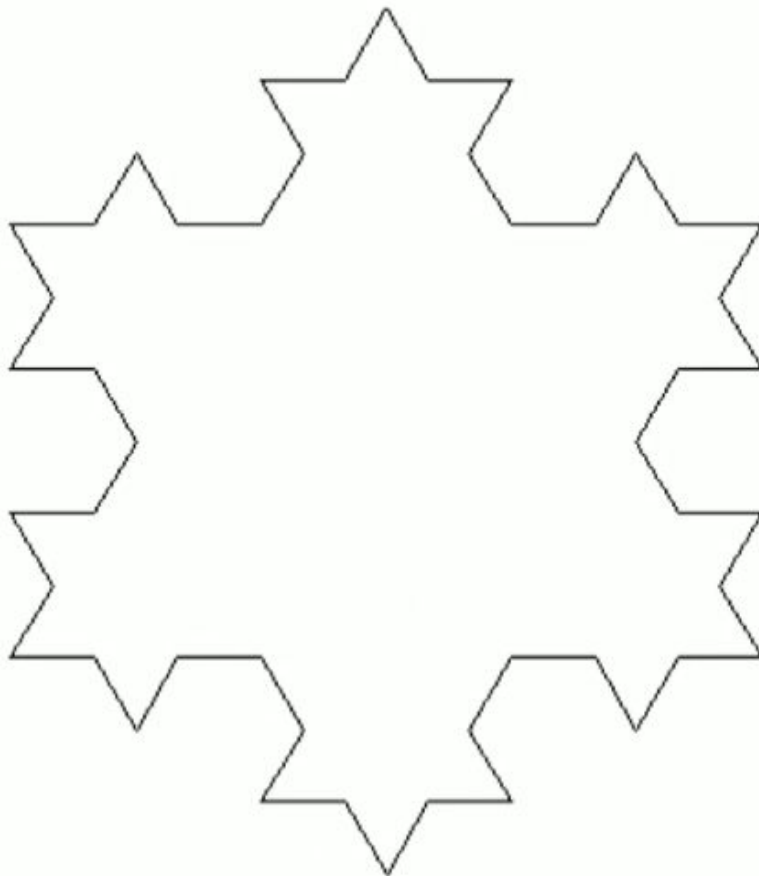
stage	no of sides	length of sides
0	3	1

The Koch Snowflake (4/7)



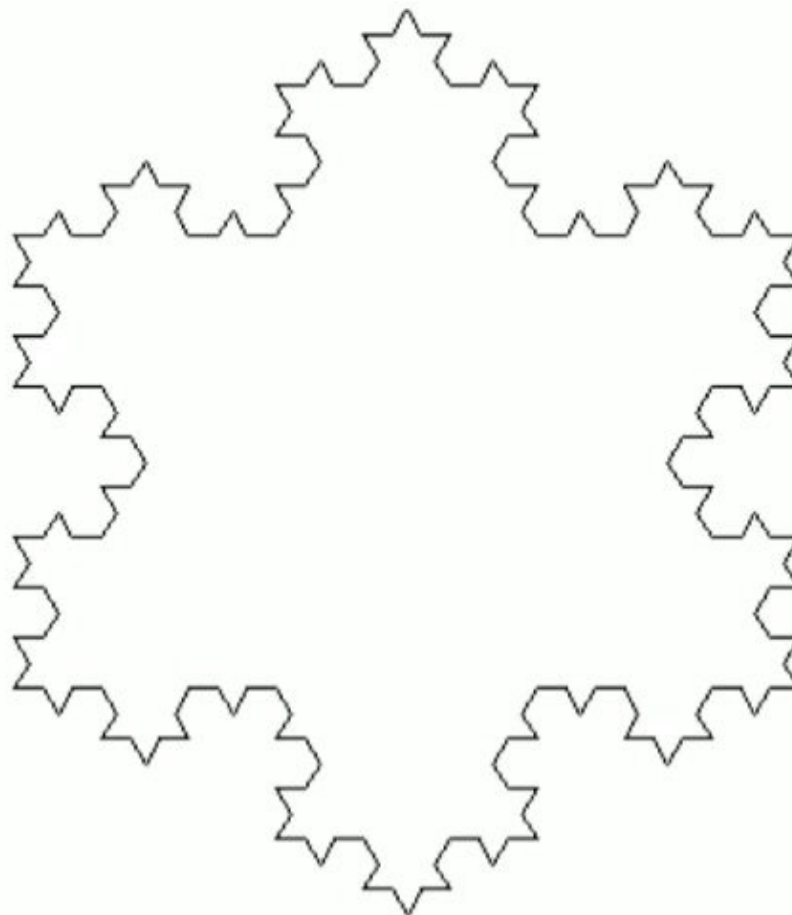
stage	no of sides	length of sides
1	3 x 4	1/3

The Koch Snowflake (5/7)



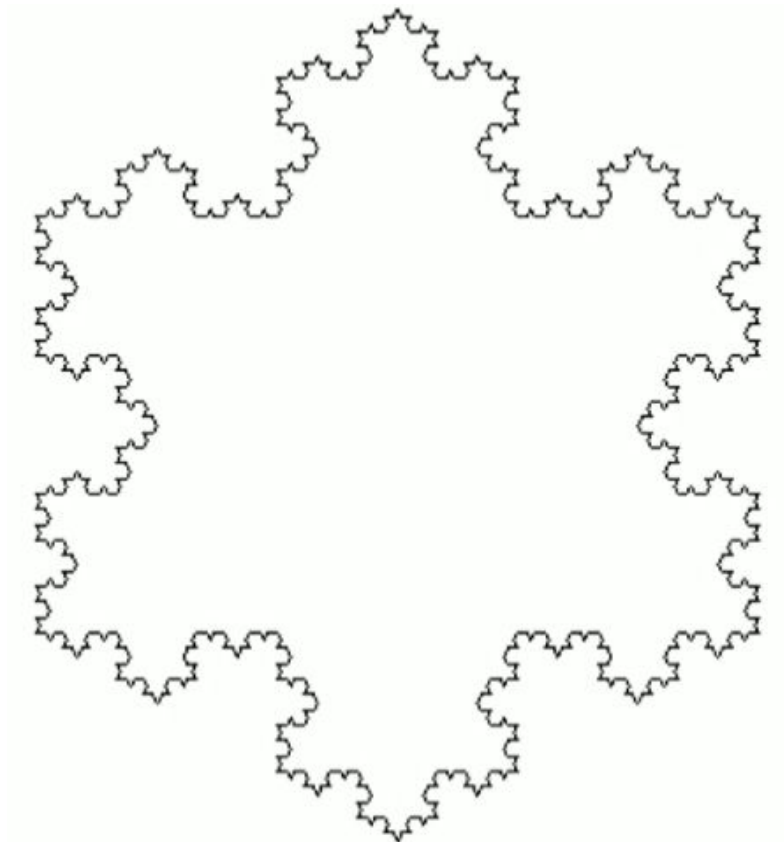
stage	no of sides	length of sides
2	3 x 4 x 4	$1/3^2$

The Koch Snowflake (6/7)



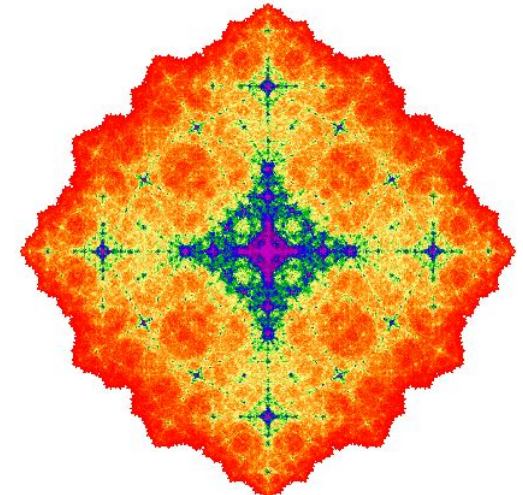
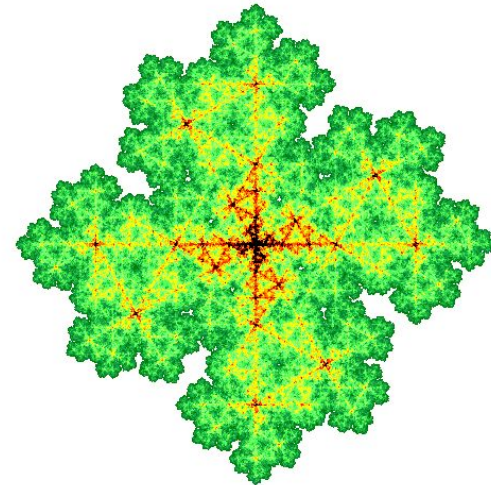
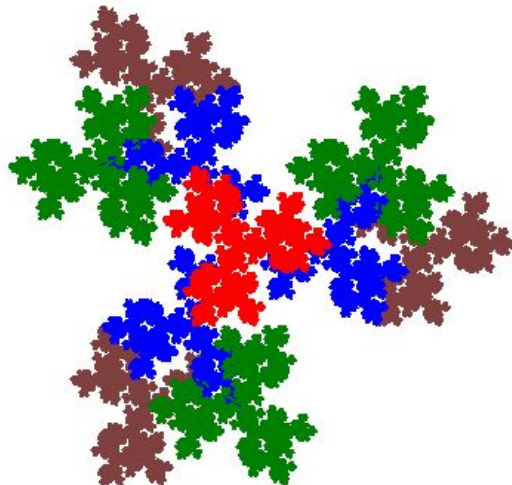
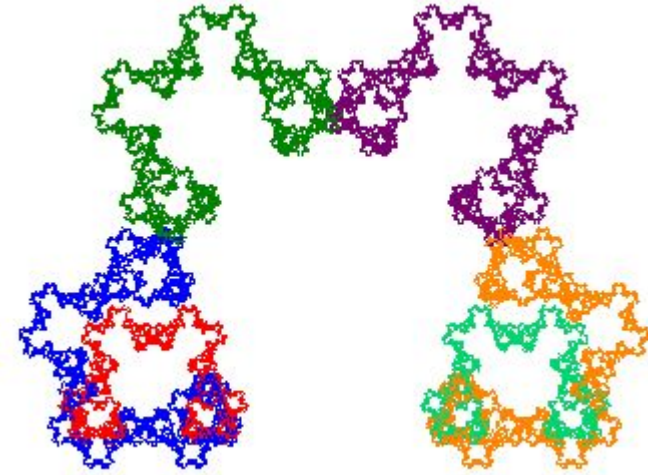
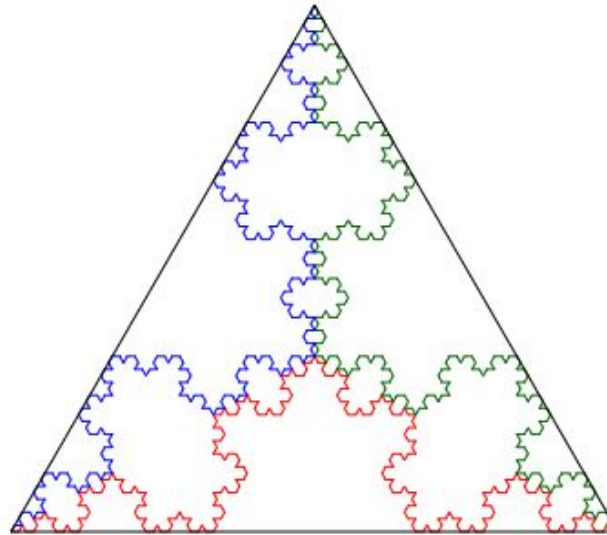
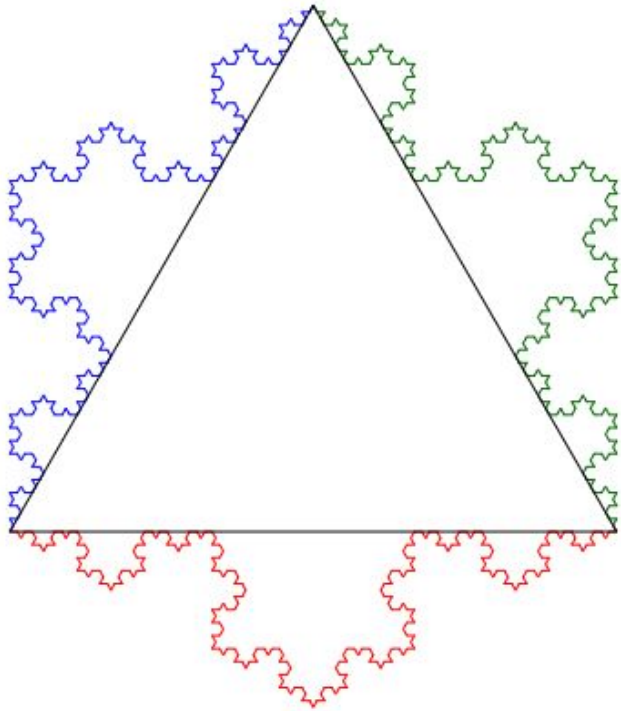
stage	no of sides	length of sides
3	$3 \times 4 \times 4 \times 4$	$1/3^3$

The Koch Snowflake (7/7)

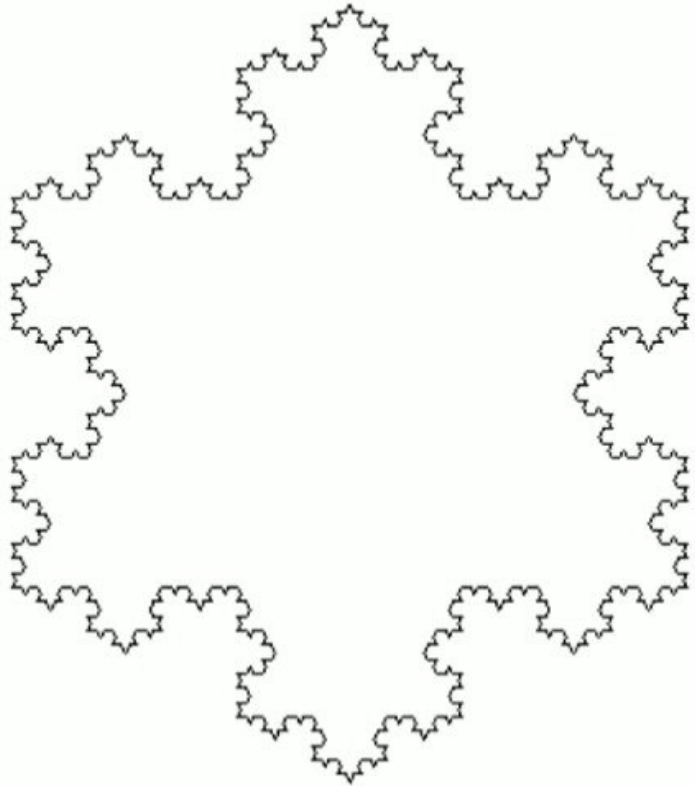


stage	no of sides	length of sides
4	$3 \times 4 \times 4 \times 4 \times 4$	$1/3^4$

Colorful Koch Snowflakes



Fractal Dimension of Koch Snowflake (1/2)



stage	no of sides	length of sides
0	3	1
1	3×4	$1/3$
2	$3 \times 4 \times 4$	$1/3^2$
3	$3 \times 4 \times 4 \times 4$	$1/3^3$
4	$3 \times 4 \times 4 \times 4 \times 4$	$1/3^4$
...	...	...
n	3×4^n	$1/3^n$

Fractal Dimension of Koch Snowflake (2/2)

Using

$$D = -\lim_{n \rightarrow \infty} \frac{\log(N_n)}{\log(\epsilon_n)}$$

then the fractal dimension of Koch Snowflake is

$$\begin{aligned} D &= -\lim_{n \rightarrow \infty} \frac{\log [3(4^n)]}{\log [\frac{1}{3^n}]} = -\lim_{n \rightarrow \infty} \frac{\log(3) + n \log 4}{-n \log (3)} \\ &= \lim_{n \rightarrow \infty} \frac{\log(3)}{n \log (3)} + \frac{n \log 4}{n \log (3)} = \frac{\log(4)}{\log(3)} \\ &= 1.2619 \dots \end{aligned}$$

Fractal Dimension of Koch Snowflake (2/2)

Using

$$D = -\lim_{n \rightarrow \infty} \frac{\log(N_n)}{\log(\epsilon_n)}$$

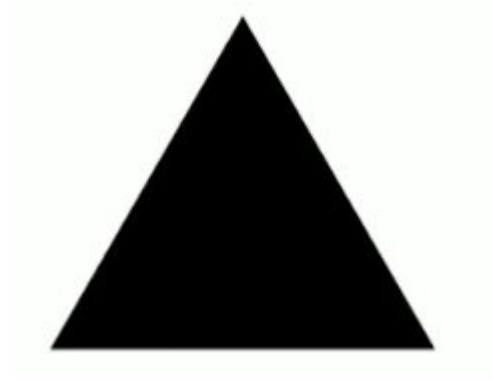
then the fractal dimension of Koch Snowflake is

$$\begin{aligned} D &= -\lim_{n \rightarrow \infty} \frac{\log [3(4^n)]}{\log [\frac{1}{3^n}]} = -\lim_{n \rightarrow \infty} \frac{\log(3) + n \log 4}{-n \log (3)} \\ &= \lim_{n \rightarrow \infty} \frac{\log(3)}{n \log (3)} + \frac{n \log 4}{n \log (3)} = \frac{\log(4)}{\log(3)} \\ &= 1.2619 \dots \end{aligned}$$

Sierpinski Triangle (1/5)

- Attributed to Wacław Sierpinski, the Sierpinski has also been seen in the medieval mosaics
- It starts with a filled equilateral triangle at stage 0 and the rules are:
 - For each filled triangle, make an unfilled copy
 - Scale the copy by half and invert it
 - Place the copy in the center of the filled triangle

Sierpinski Triangle (2/5)



stage 0

Sierpinski Triangle (3/5)



stage 0



stage 1

Sierpinski Triangle (4/5)



stage 0



stage 1



stage 2

Sierpinski Triangle (5/5)



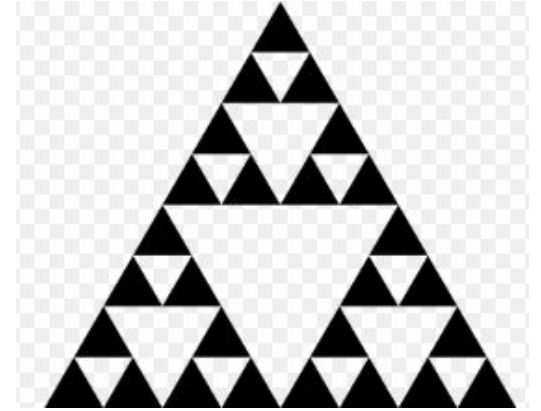
stage 0



stage 1



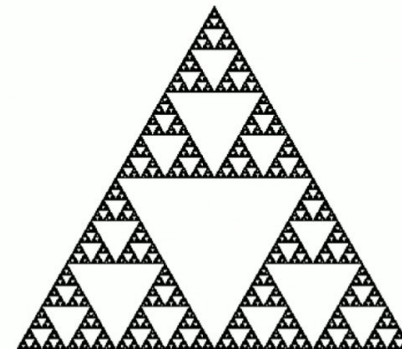
stage 2



stage 3

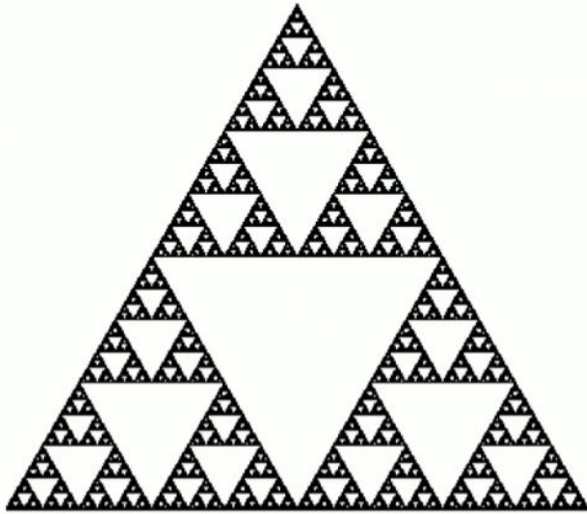


stage 5



stage 6

Fractal Dimension of Sierpinski Triangle



$$D = \frac{\log(3)}{\log(2)}$$
$$= 1.585 \dots$$

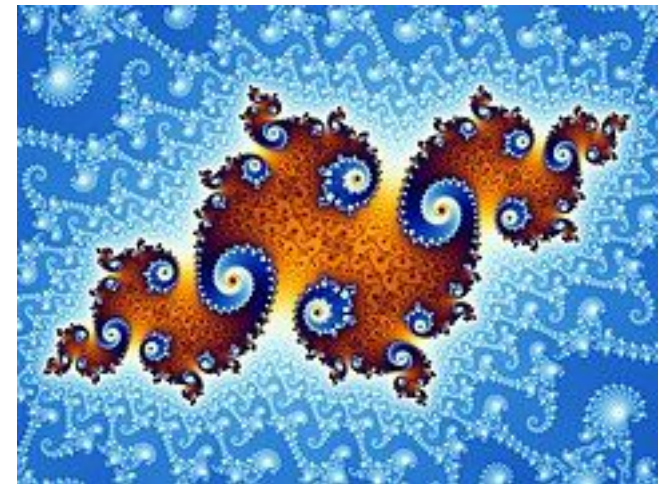
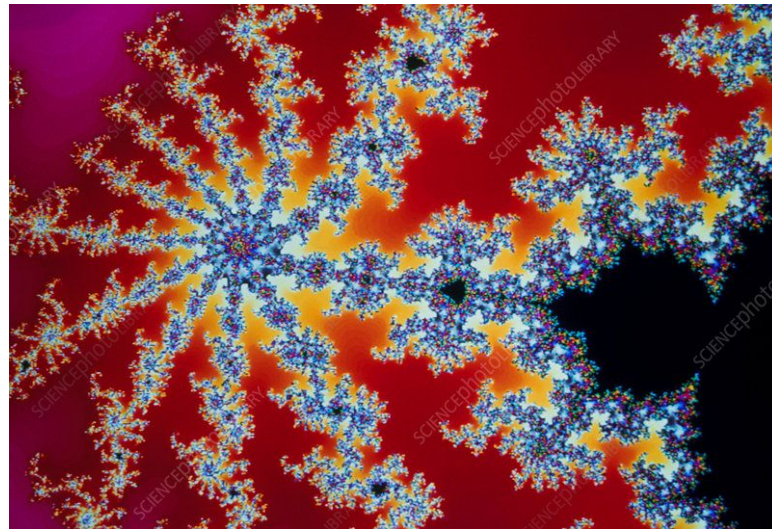
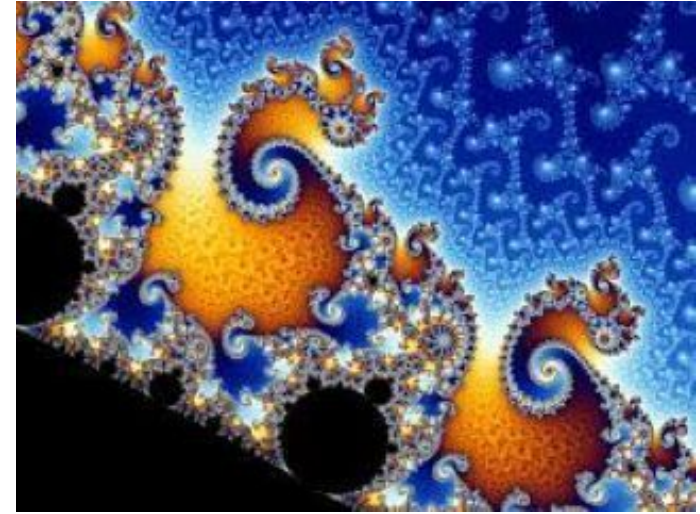
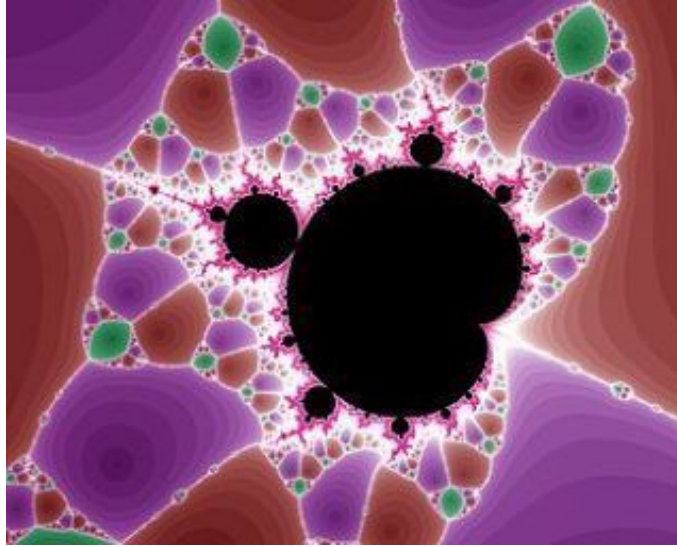
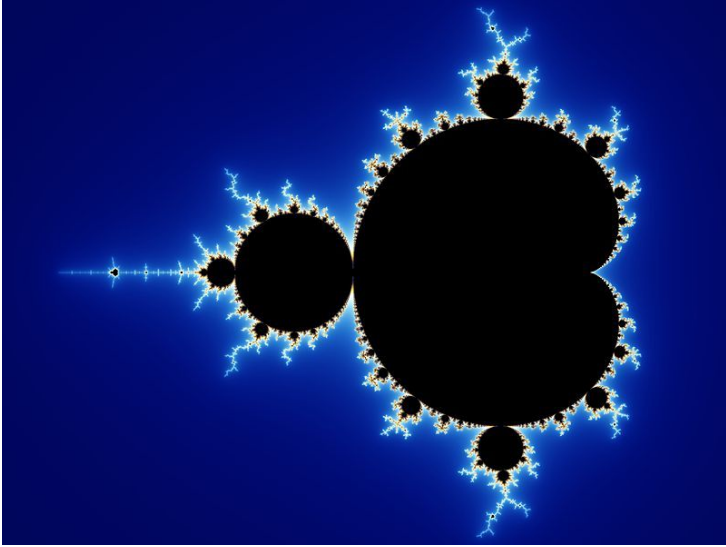
How?

[Zoom into Sierpinski Triangle](#)

The Mandelbrot Set

- The most famous fractal is the **Mandelbrot set** by Benoit Mandelbrot
- It is a very very beautiful and complex shape that is generated from a iterative scheme
- The Mandelbrot set can be generated by a simple iterative formula, called quadratic recurrence relation

The Mandelbrot Set



Complex Numbers

Complex Numbers:

- The Mandelbrot set is defined using the complex numbers
- A complex number z is defined using the number of the form:

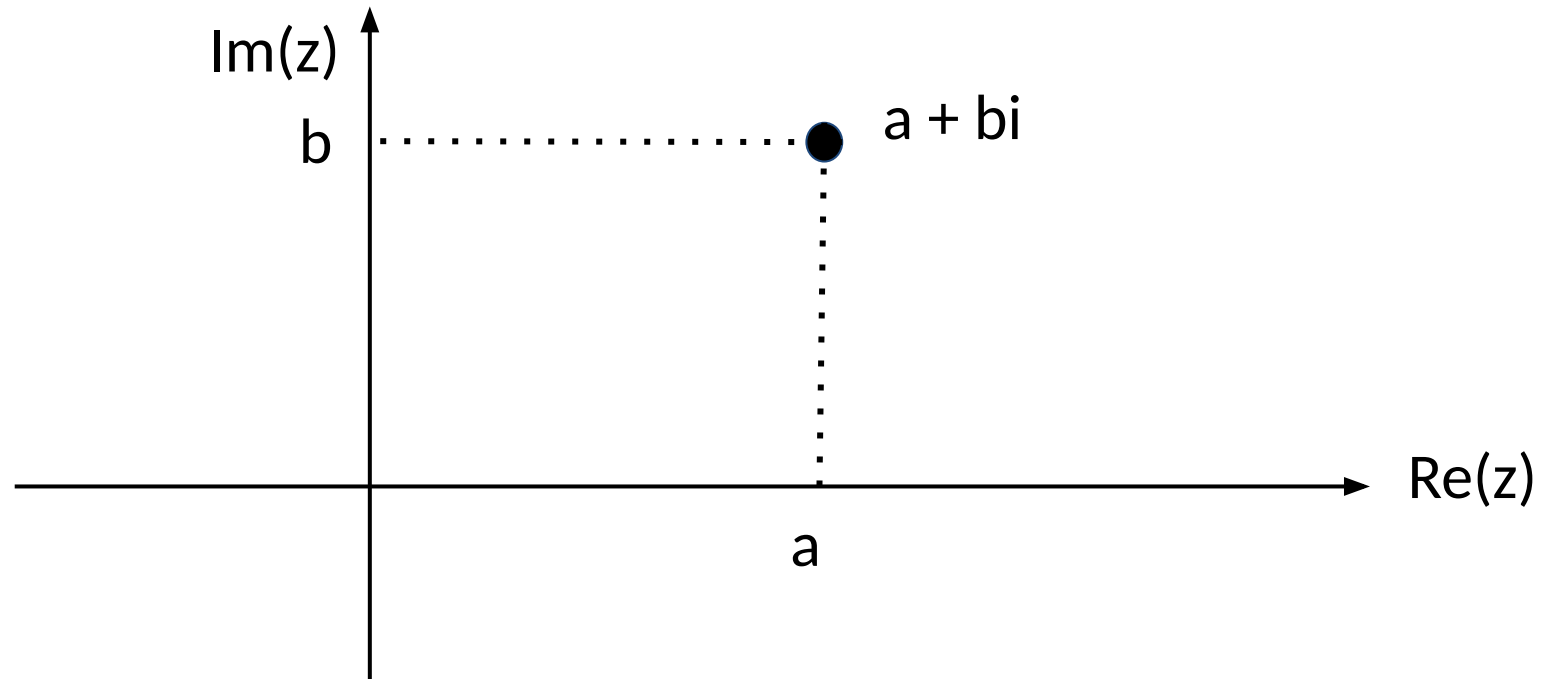
$$z = a + bi$$

where $a, b \in \mathbb{R}$ and $i^2 = -1$

- a is known as the real part of z and is denoted by $\text{Re}(z)$
- b is known as the imaginary part of z and is denoted by $\text{Im}(z)$

Complex Numbers

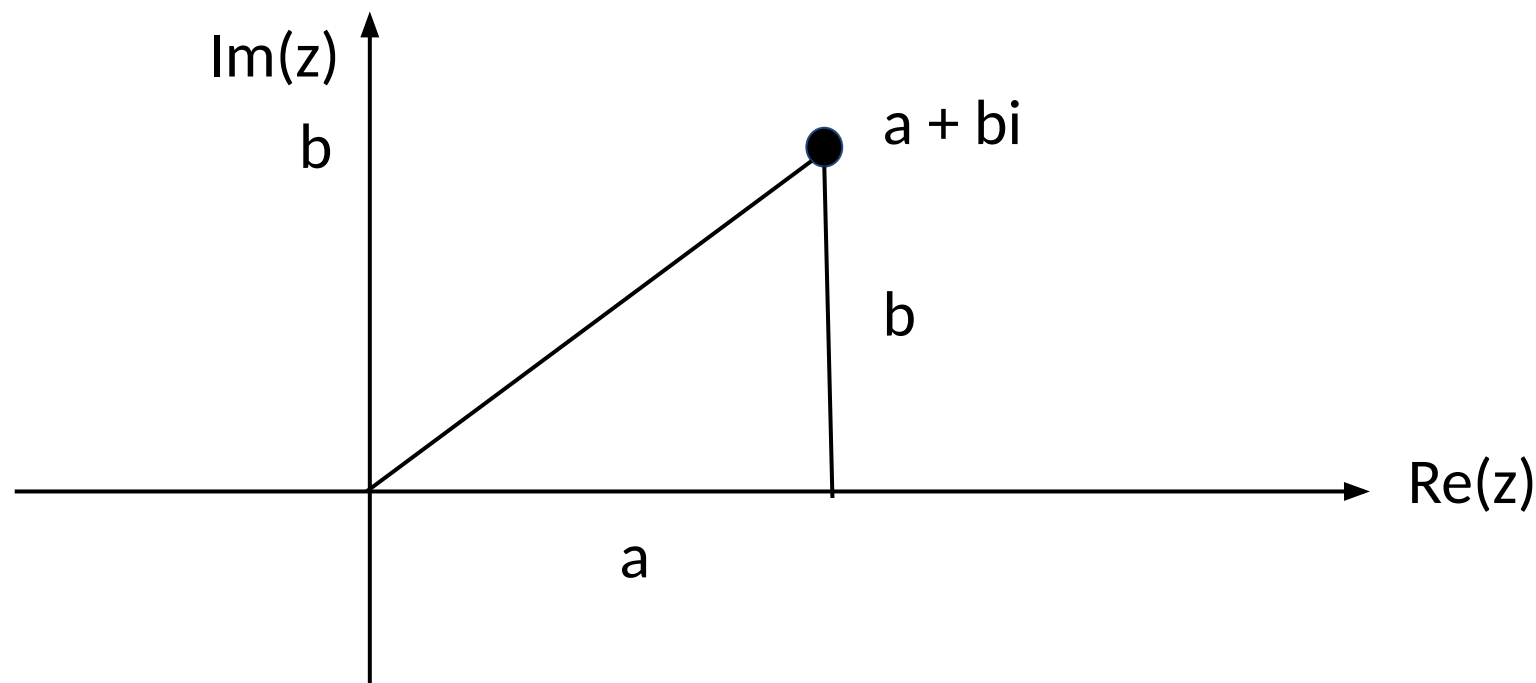
- Complex number can be plotted on a set of axes where the horizontal axes represent real part and the vertical axes represent imaginary part
- The axes are known as **complex plane**



Complex Numbers

- The modulus of a complex number, $|a + bi|$, is the distance between the complex number and the origin

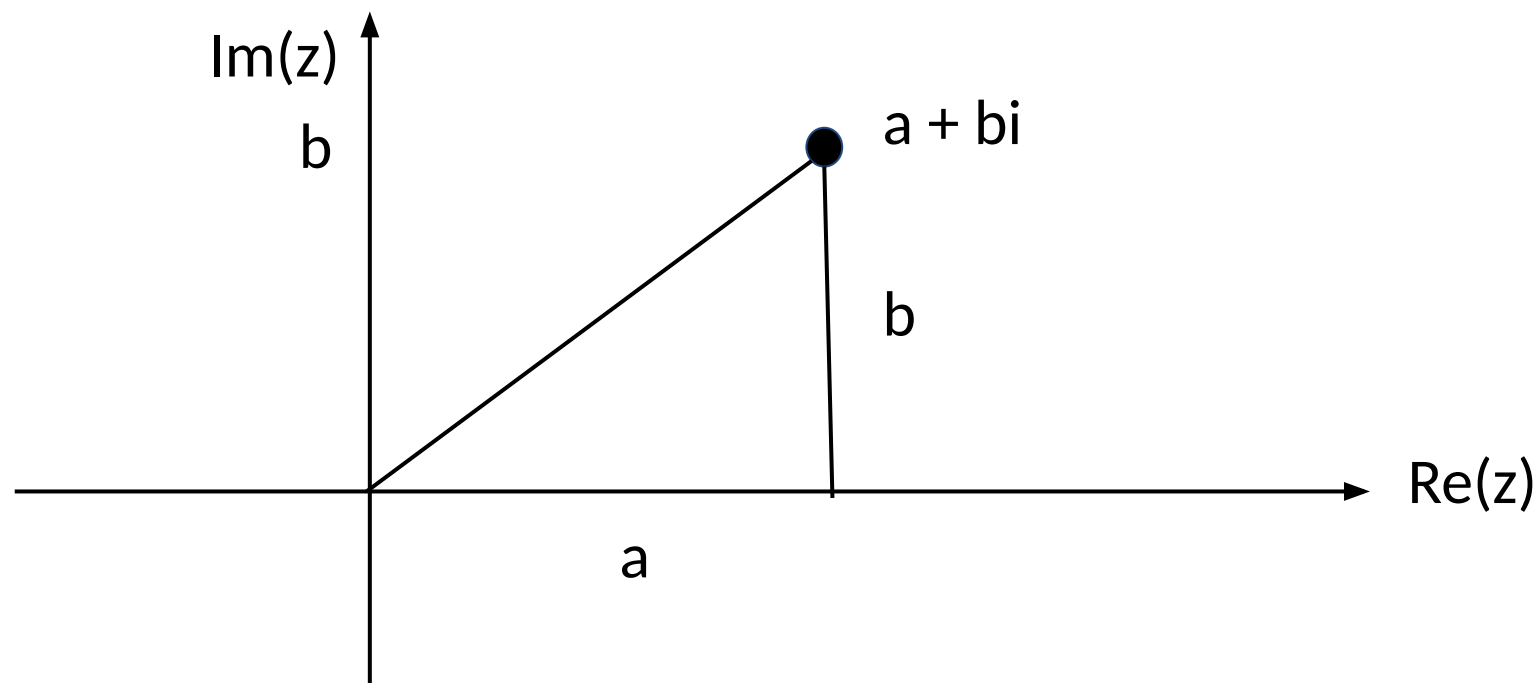
$$|a + bi| = \sqrt{a^2 + b^2}$$



Complex Numbers

- The modulus of a complex number, $|a + bi|$, is the distance between the complex number and the origin

$$|a + bi| = \sqrt{a^2 + b^2}$$



Generating Mandelbrot Set (1/8)

- The Mandelbrot set is defined as a set of points in the complex plane for which the iterative scheme:

$$z_{n+1} = z_n^2 + c$$

remains bounded, i.e. $|z_n|$ does not tends to infinity

- c is a complex number
- $z_0 = 0$ is used for starting value
- A point c is said to have escaped (therefore not the member of Mandelbrot set) if z_n is larger than some escape radius (usually 2)

Generating Mandelbrot Set (2/8)

iterative scheme:

$$z_{n+1} = z_n^2 + c$$

- Consider the iterations for $c = 1 + 0i$

$z_0 = 1 + 0i$	$ z_0 = 1$
$z_1 = 1^2 + 1 + 0i = 1 + 1 = 2$	$ z_1 = 2$
$z_2 = 2^2 + 1 + 0i = 4 + 1 = 5$	$ z_2 = 5$
$z_3 = 5^2 + 1 + 0i = 25 + 1 = 26$	$ z_3 = 26$
$z_4 = 26^2 + 1 + 0i = 1 + 1 = 677$	$ z_4 = 677$

- Therefore $c = 1 + 0i$ is not in the Mandelbrot set

Generating Mandelbrot Set (3/8)

iterative scheme:

$$z_{n+1} = z_n^2 + c$$

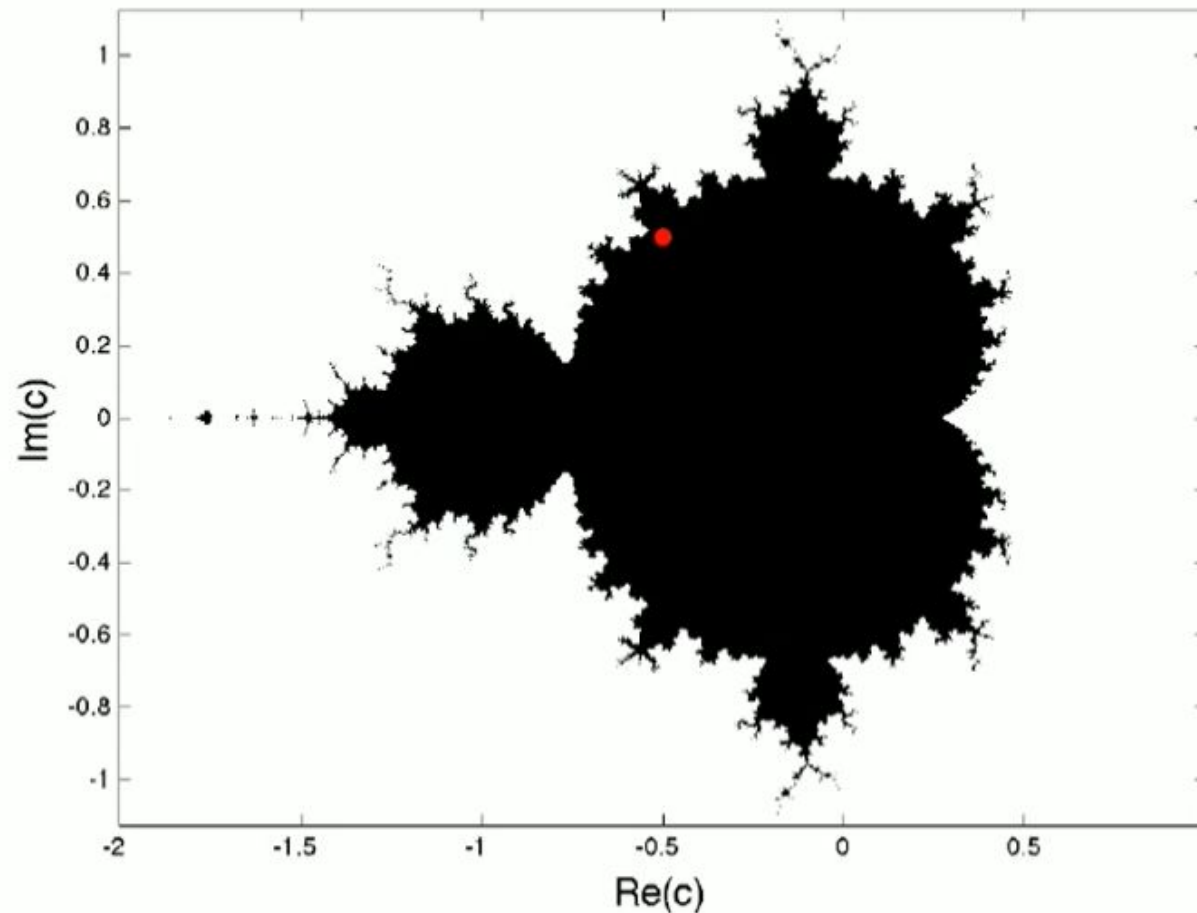
- Consider the iterations for $c = 0 + i$

$z_0 = 0 + i$	$ z_0 = -1$
$z_1 = (0 + i)^2 + 0 + i = -1 + i$	$ z_1 = \sqrt{2}$
$z_2 = (-1 + i)^2 + 0 + i = -i$	$ z_2 = 1$
$z_3 = (0 - i)^2 + 0 + i = -1 + i$	$ z_3 = \sqrt{2}$
$z_4 = (-1 + i)^2 + 0 + i = -i$	$ z_4 = 1$

- Therefore $c = 0 + i$ is in the Mandelbrot set

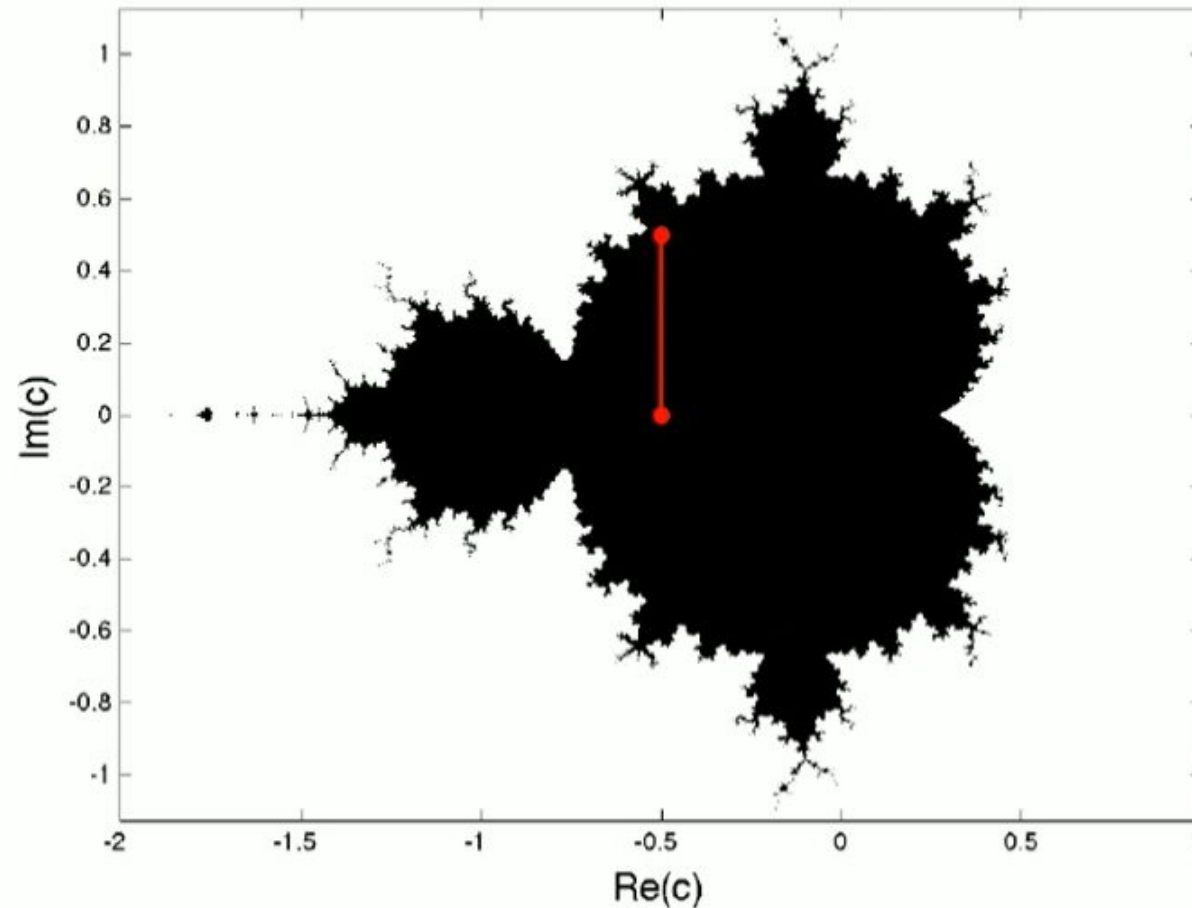
Generating Mandelbrot Set (4/8)

Convergence behaviour of $c = 0.5 + 0.5i$



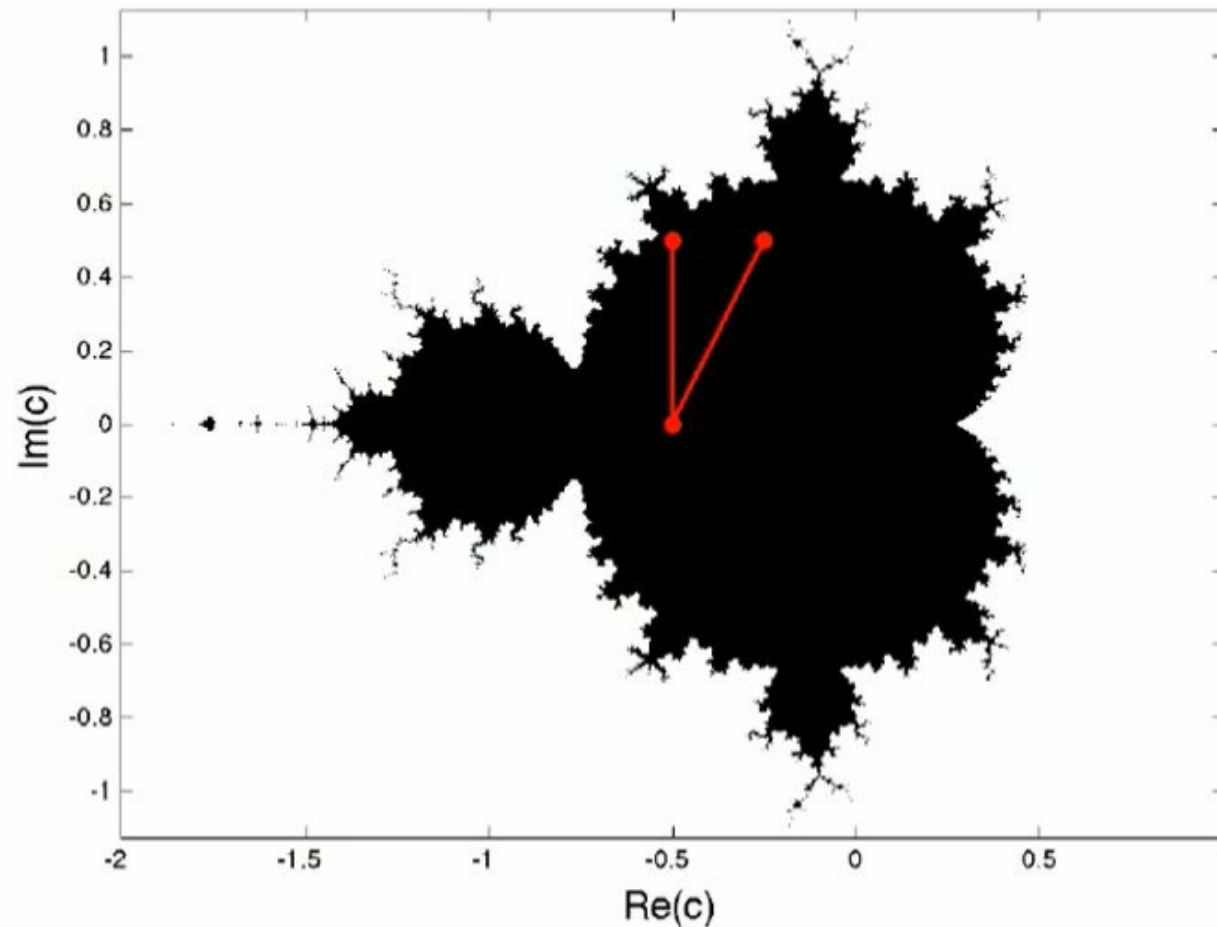
Generating Mandelbrot Set (5/8)

Convergence behaviour of $c = 0.5 + 0.5i$



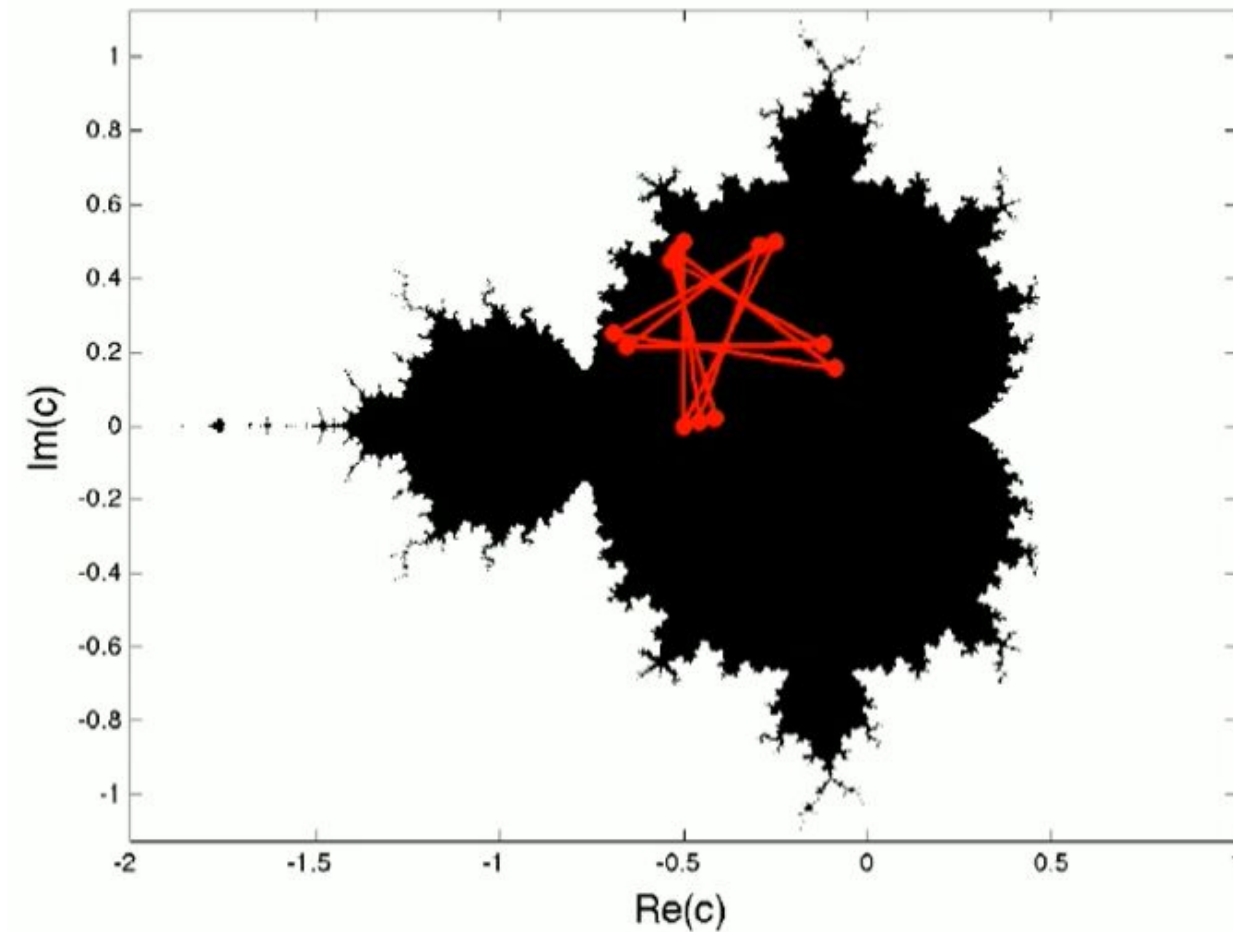
Generating Mandelbrot Set (6/8)

Convergence behaviour of $c = 0.5 + 0.5i$



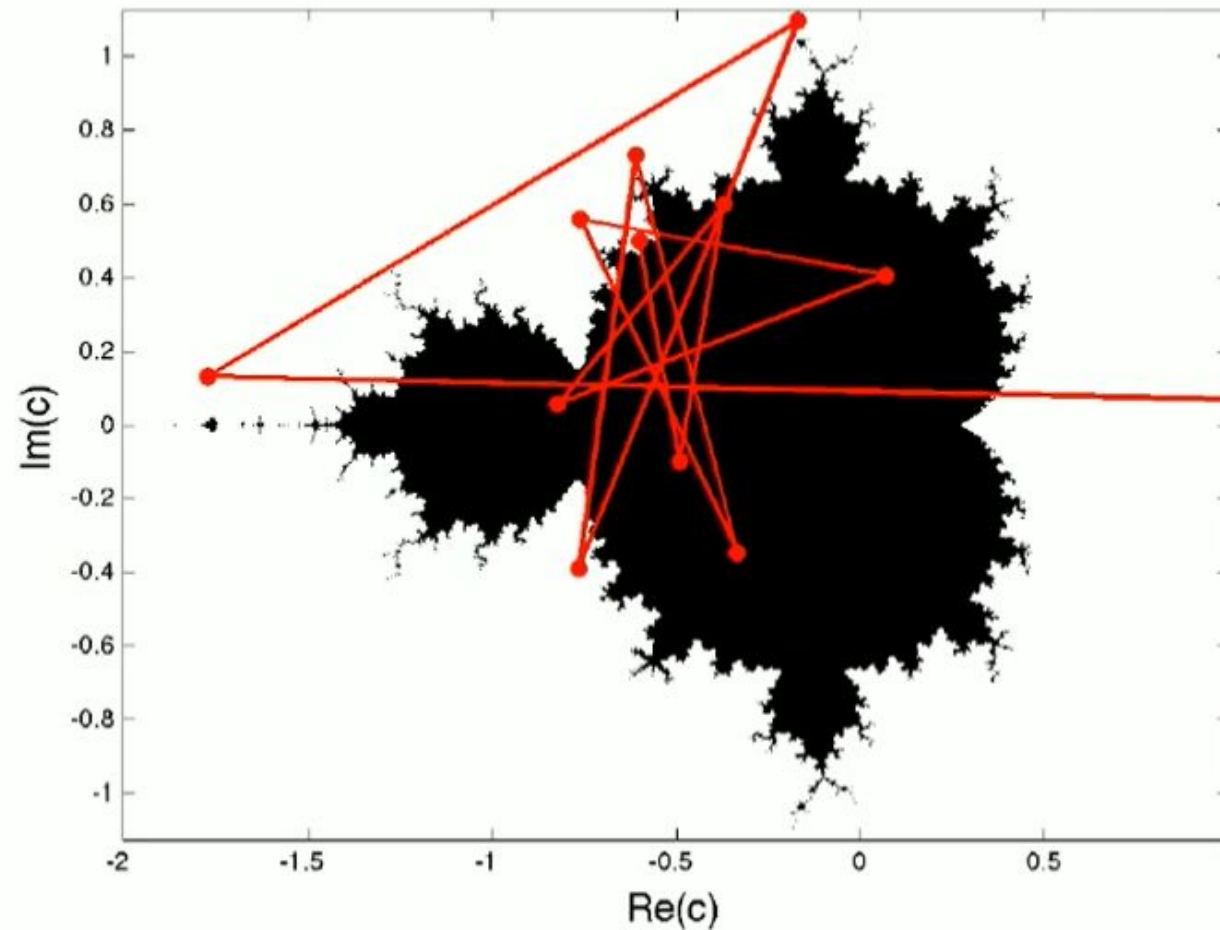
Generating Mandelbrot Set (7/8)

Convergence behaviour of $c = 0.5 + 0.5i$



Generating Mandelbrot Set (8/8)

Convergence behaviour of $c = -0.6 + 0.5i$

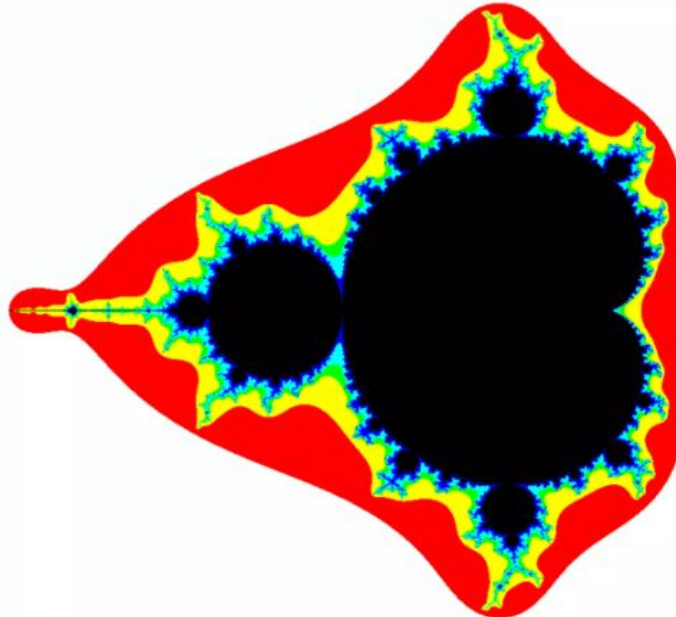


Mandelbrot Set using Color

- Our Mandelbrot set do not plot points that aren't in the set
- We can color these points depending on the number of iterations to divergence

Red = z_5 , Yellow = z_8 , Green = z_{12}
Light Blue = z_{15} , Dark Blue = z_{25} , Black = z_{100} .

[zoom into Mandelbrot Set](#)



Further Study

- <https://www.youtube.com/watch?v=bEXW7V9mATU>
- https://www.youtube.com/watch?v=NGMRB4O922I&ab_channel=Numberphile

CSE4203: Computer Graphics
Lecture – 8 (Part - B)
Fractal Geometry

Outline

- L-systems

L-systems

- Lindenmayer-systems or L-system is developed by 1968 by biologist Aristid Lindenmayer
- It's a grammar based technique
 - Represent shape as string of symbol
 - Each symbol has meaning in drawing shape
- Two parts:
 - Grammar for generating strings
 - Rendering algorithm for interpreting strings as shapes

L-system structure

- L-systems are commonly known as parametric L systems, defined as a tuple

$$\mathbf{G} = (V, \omega, P)$$

- **V (the alphabet)** is a set of symbols containing both elements that can be replaced (variables) and those which cannot be replaced ("constants" or "terminals")
- **ω (start, axiom or initiator)** is a string of symbols from V defining the initial state of the system
- **P is a set of production rules** or productions defining the way variables can be replaced with combinations of constants and other variables.

Example - 1: algae (1/4)

- L-system for modelling the growth of algae
variables : **A B**
constants : **none**
axiom : **A**
rules : **(A $\rightarrow$ AB), (B $\rightarrow$ A)**

Example - 1: algae (2/4)

$n = 0 : A$

$n = 1 : AB$

$n = 2 : ABA$

$n = 3 : ABAAB$

$n = 4 : ABAABABA$

$n = 5 : ABAABABAABAAB$

$n = 6 : ABAABABAABAABAABAABA$

$n = 7 : ABAABABAABAABAABAABAABAABAABAABAABAABA$

axiom : A

rules : $(A \rightarrow AB)$, $(B \rightarrow A)$

Example - 1: algae (3/4)

axiom : A
rules : (A $\rightarrow$ AB), (B $\rightarrow$ A)

n=0: A start (axiom/initiator)

 / \

n=1: A B the initial single A spawned into AB by rule (A $\rightarrow$ AB), rule (B $\rightarrow$ A)

couldn't be applied

 / | \

n=2: A B A former string AB with all rules applied, A spawned into AB again,

former B turned into A

 / | | | \

n=3: A B A A B note all A's producing a copy of themselves in the first place, then

a B, which turns ...

 / | | | \ | \ \

n=4: A B A A B A B A ... into an A one generation later, starting to spawn/repeat/recurse

then

Example - 1: algae (4/4)

axiom : A
rules : (A → AB), (B → A)

n=0: A start (axiom/initiator)

 / \

n=1: A B the initial single A spawned into AB by rule (A → AB), rule (B → A)

couldn't be applied

 / | \

n=2: A B A former string AB with all rules applied, A spawned into AB again,

former B turned into A

 / | | | \

n=3: A B A A B note all A's producing a copy of themselves in the first place, then

a B, which turns ...

 / | | | | \ \

n=4: A B A A B A B A ... into an A one generation later, starting to spawn/repeat/recurse

If we count the length of each string, we obtain the Fibonacci sequence of numbers: 1 2 3 5 8 13 21 34 55 89 ...

Example - 2: Cantor set (1/6)

variables : **A B**

constants : **none**

axiom : **A**

rules : **(A $\rightarrow$ ABA), (B $\rightarrow$ BBB)**

A: Draw a line forward

B: Move forward without drawing

Example - 2: Cantor set (2/6)

n = 0: A

n = 1: ABA

n = 2: ABABBBABA

n = 3: ABABBBABABBBBBBBBBBABABBBABA

variables : A B

constants : none

axiom : A

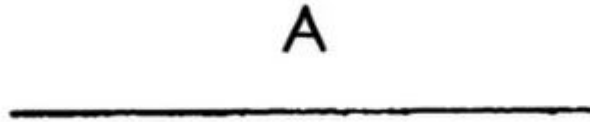
rules : (A $\rightarrow$ ABA), (B $\rightarrow$ BBB)

A: Draw a line forward

B: Move forward without drawing

Example - 2: Cantor set (3/6)

$n = 0$: A



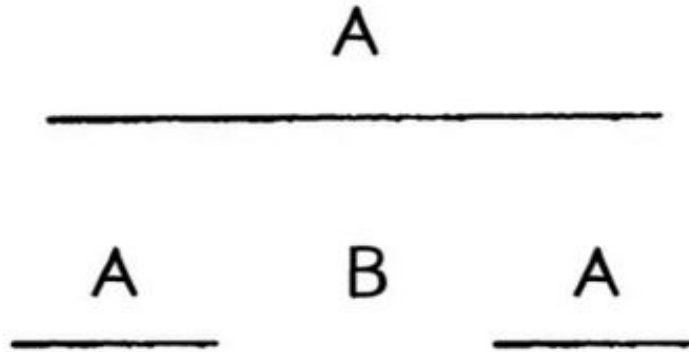
A: Draw a line forward

B: Move forward without drawing

Example - 2: Cantor set (4/6)

n = 1: ABA

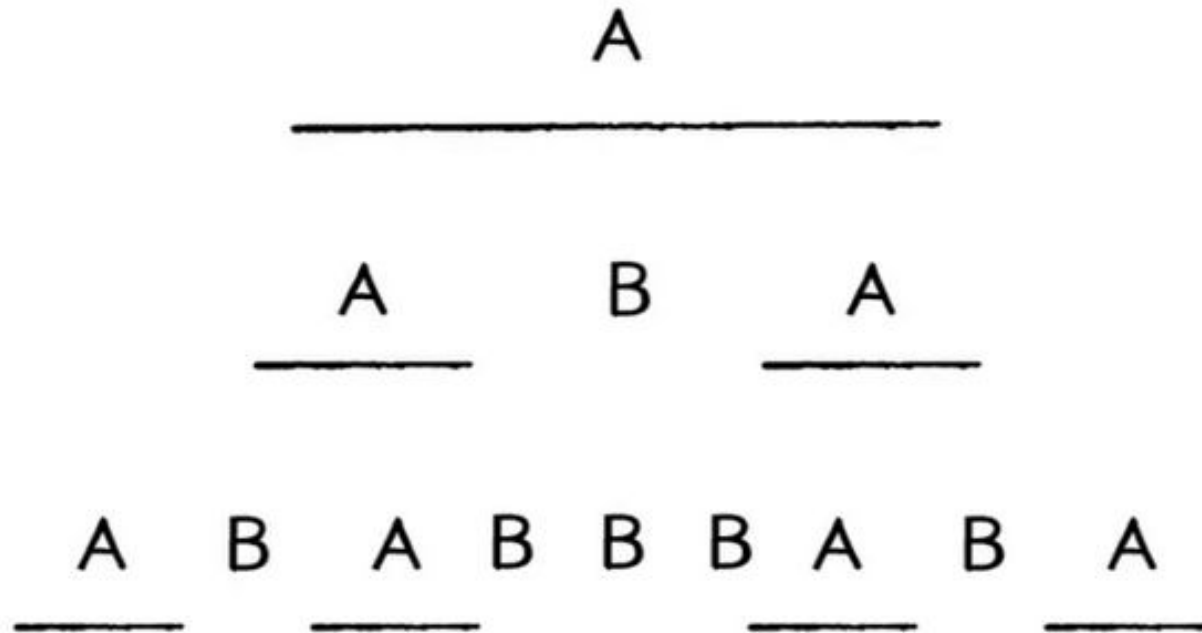
A: Draw a line forward
B: Move forward without drawing



Example - 2: Cantor set (5/6)

n = 2: ABABBBABA

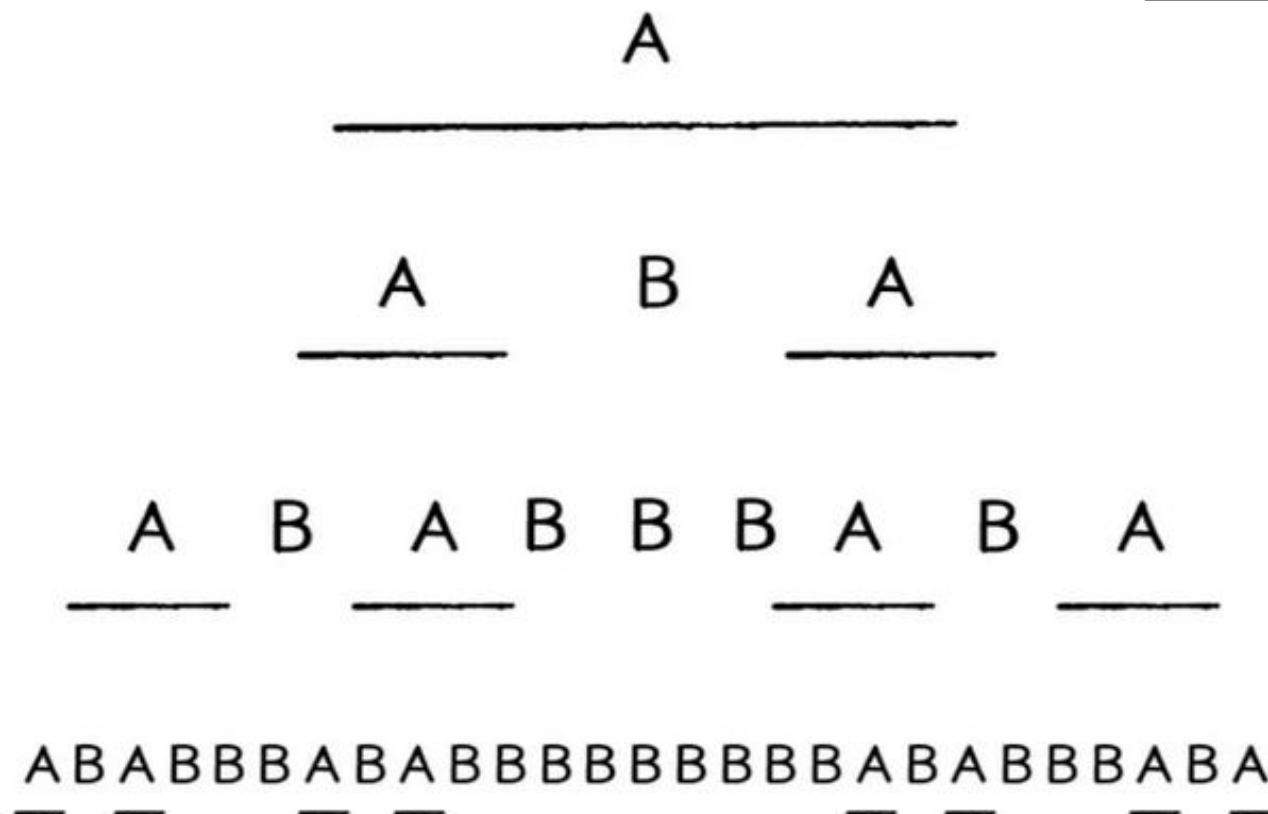
A: Draw a line forward
B: Move forward without drawing



Example - 2: Cantor set (6/6)

n = 3: ABABBBBABABBBBBBBBBBABABBBABA

A: Draw a line forward
B: Move forward without drawing



Example - 3: Koch Snowflake (1/5)

variables : **F**

constants : **+**, **-**

axiom : **F**

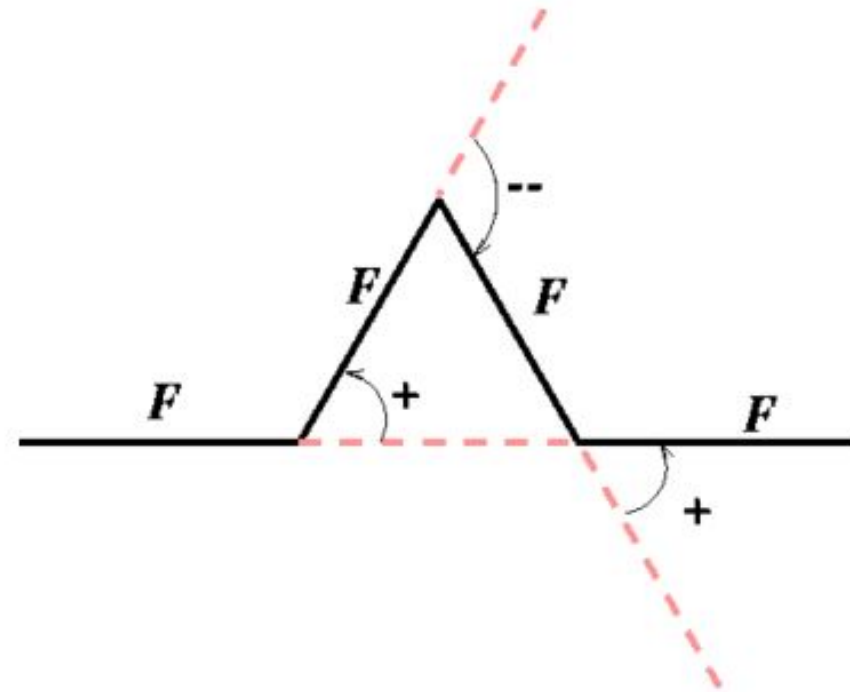
rules : **F** $\rightarrow$ **F+F--F+F**

Angle: 60 degrees

F: forward one unit

+ : turn left 60 degrees

- : turn right 60 degrees



Example - 3: Koch Snowflake (2/5)

n = 0: F

n = 1: F+F--F+F

n = 2: F+F--F+FF+F--F+FF+F--F+FF+F--F+F

variables : F, +, -

constants : none

axiom : F

rules : F -> F+F--F+F

Angle: 60 degrees

F: forward one unit

+ : turn left 60 degrees

- : turn right 60 degrees

Example - 3: Koch Snowflake (3/5)

n = 0: F

F: forward one unit
+ : turn left 60 degrees
- : turn right 60 degrees



Example - 3: Koch Snowflake (4/5)

$n = 1$: F+F--F+F

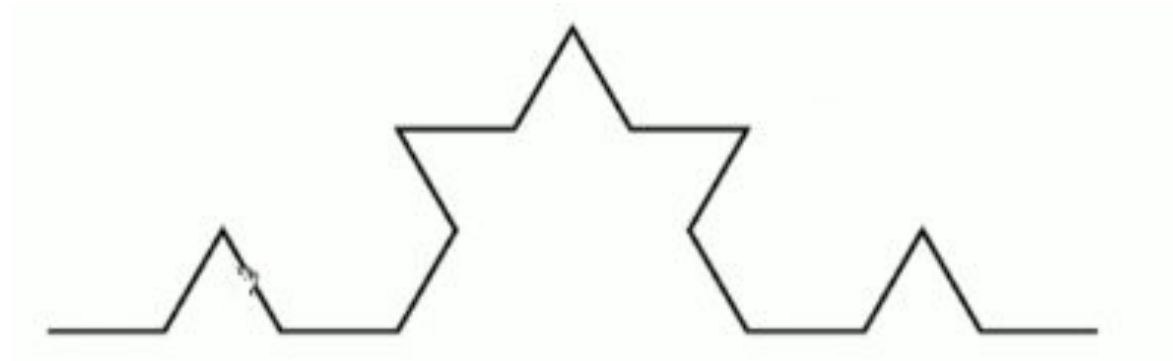
F: forward one unit
+: turn left 60 degrees
-: turn right 60 degrees



Example - 3: Koch Snowflake (5/5)

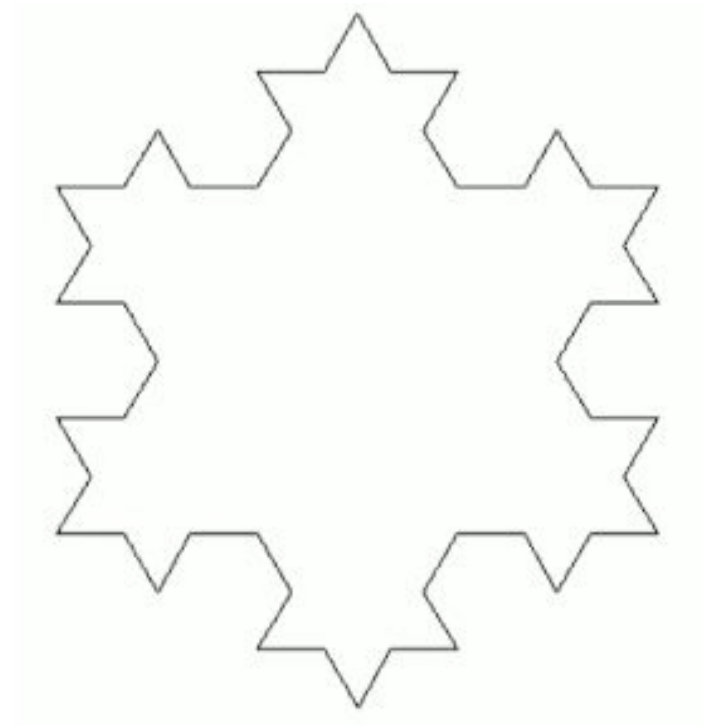
$n = 2$: F+F--F+F+F+F--F+F--F+F--F+F+F+F--F+F

F: forward one unit
+: turn left 60 degrees
-: turn right 60 degrees



Example - 3: Koch Snowflake (5/5)

For all 3 sides of triangle



Example - 4: Fractal (binary) tree (1/10)

variables : **0 1**

constants : **[]**

axiom : **0**

rules : **(1 → 11), (0 → 1[0]0)**

Angle: 45 degrees

0: draw a line segment ending in a leaf

1: draw a line segment

[: push position and angle, turn left 45 degrees

]: pop position and angle, turn right 45 degrees

Example - 4: Fractal (binary) tree (2/10)

n = 0: 0

n = 1: 1[0]0

n = 2: 11[1[0]0]1[0]0

n = 3: 1111[11[1[0]0]1[0]0]11[1[0]0]1[0]0

variables : 0 1

constants : []

axiom : 0

rules : (1 → 11), (0 → 1[0]0)

Angle: 45 degrees

0: draw a line segment ending in a leaf

1: draw a line segment

[: push position and angle, turn left 45 degrees

] : pop position and angle, turn right 45 degrees

Example - 4: Fractal (binary) tree (3/10)

n = 0: 0

0: draw a line segment ending in a leaf



Example - 4: Fractal (binary) tree (4/10)

n = 1: 1[0]0

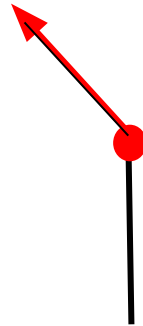
1: draw a line segment



Example - 4: Fractal (binary) tree (5/10)

n = 1: 1[0]0

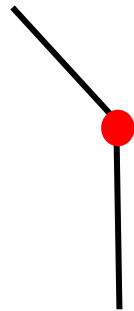
[: push position and angle, turn left 45 degrees



Example - 4: Fractal (binary) tree (6/10)

n = 1: 1[0]0

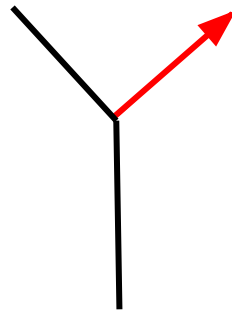
0: draw a line segment ending in a leaf



Example - 4: Fractal (binary) tree (7/10)

n = 1: 1[0]0

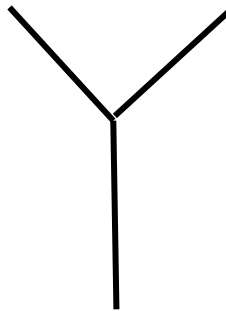
] : pop position and angle, turn right 45 degrees



Example - 4: Fractal (binary) tree (8/10)

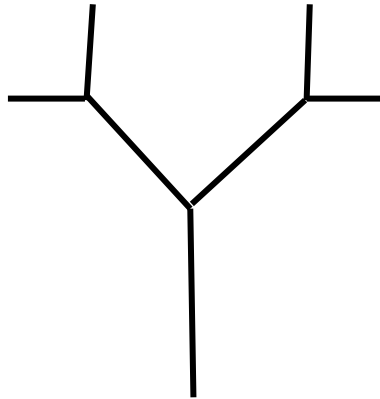
n = 1: 1[0]0

0: draw a line segment ending in a leaf



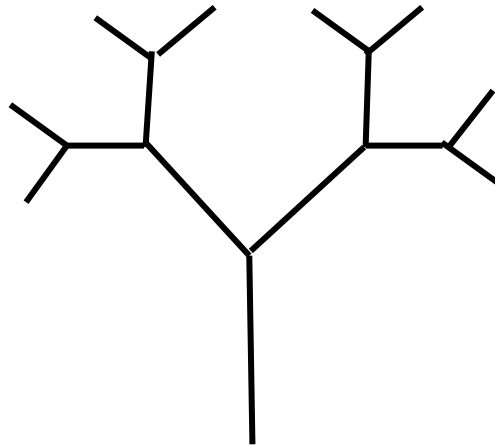
Example - 4: Fractal (binary) tree (9/10)

n = 2: 11[1[0]0]1[0]0



Example - 4: Fractal (binary) tree (10/10)

n = 3: 1111[11[1[0]0]1[0]0]11[1[0]0]1[0]0



Example - 5: Fractal Plant

variables : **X F**

constants : **+ - []**

axiom : **X**

rules : **(X $\rightarrow$ F+[[X]-X]-F[-FX]+X), (F $\rightarrow$ FF)**

Angle: 25 degrees

F: draw forward

X: do Nothing

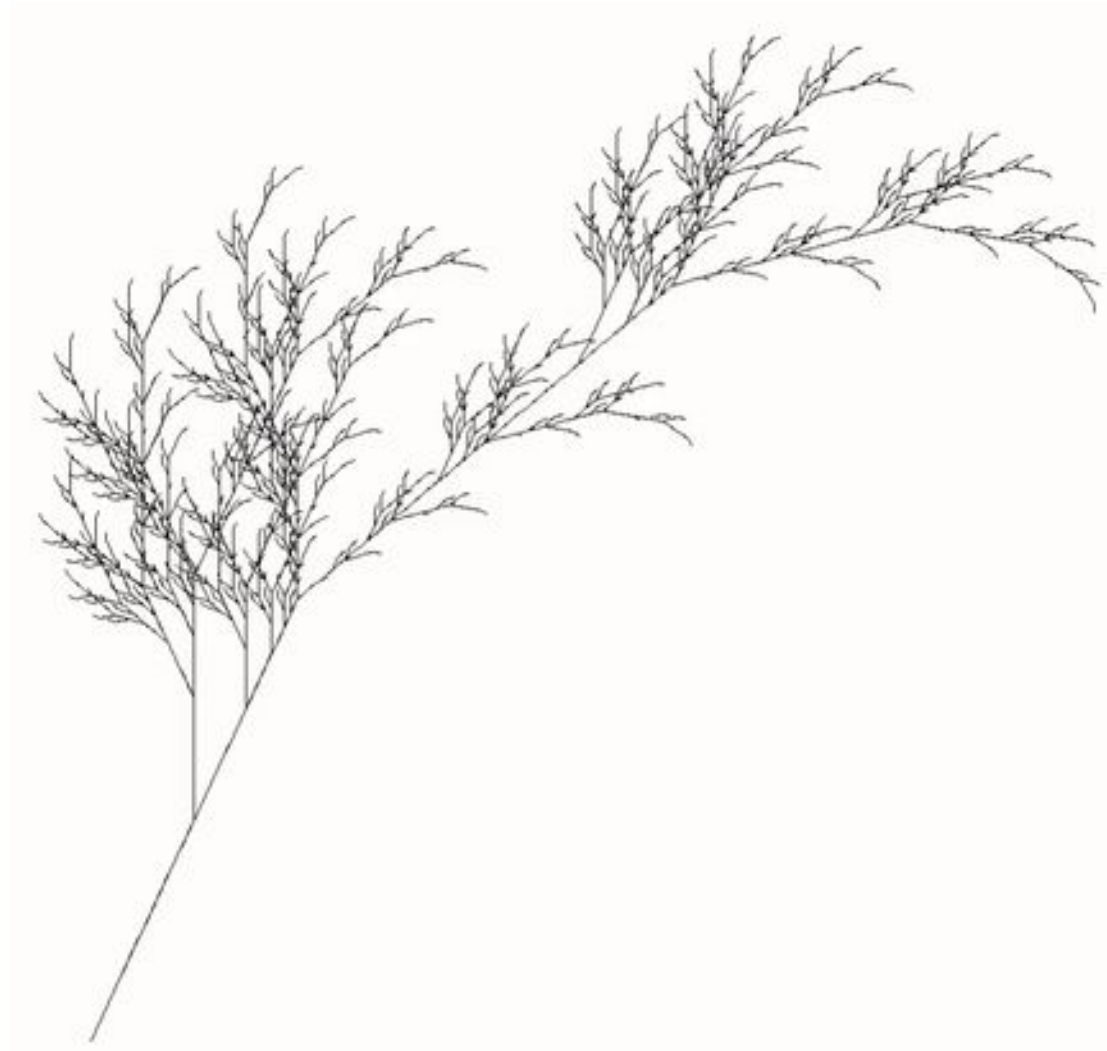
+ : turn left 25 degrees

- : turn right 25 degrees

[: push state

] : pop state

Example - 5: Fractal Plant



Fractal plant for $n = 6$ (GIF available [here](#))

Example - 5: Fractal Plant



a
 $n=5, \delta=25.7^\circ$
 F
 $F \rightarrow F[+F]F[-F]F$



b
 $n=5, \delta=20^\circ$
 F
 $F \rightarrow F[+F]F[-F][F]$



c
 $n=4, \delta=22.5^\circ$
 F
 $F \rightarrow FF - [-F + F + F] +$
 $[+F - F - F]$



d
 $n=7, \delta=20^\circ$
 X
 $X \rightarrow F[+X]F[-X] + X$
 $F \rightarrow FF$



e
 $n=7, \delta=25.7^\circ$
 X
 $X \rightarrow F[+X][-X]FX$
 $F \rightarrow FF$



f
 $n=5, \delta=22.5^\circ$
 X
 $X \rightarrow F - [[X] + X] + F[+FX] - X$
 $F \rightarrow FF$

Practice Problems

Apply the concept of L-systems for third iteration ($n = 3$) to draw -

- Sierpinski Triangle
- Koch curve
- Dragon curve

Further Reading

- <https://www.wikiwand.com/en/L-system>
- <http://paulbourke.net/fractals/lsys/>